A Single-Cycle RISC-V (RV32I Subset) Processor in VHDL: Design, Verification and FPGA Implementation

Clara Ureche
BEng (Hons) Electrical and Electronic Engineering, Lancaster University

Report type	Technical report
Status	Simulated, synthesised and run on an FPGA development board
Tools	VHDL, EDA Playground, FPGA vendor synthesis and place-and-route flow
Date	2026

Abstract. A single-cycle soft-core processor implementing a subset of the RISC-V RV32I integer instruction set was written in register-transfer-level VHDL. The datapath comprises a program counter, instruction memory, a combinational decoder, a 32 x 32-bit register file with two read ports and one write port (x0 hardwired to zero), an ALU with a zero flag for branch resolution, and a write-back path. Verification was performed in two stages: cycle-level testbench simulation of the register file, ALU and instruction sequencing, followed by hand-assembled programs whose final register state was checked against expected values. The design was then synthesised, placed and routed, and executed the test program on an FPGA board. Resource utilisation and maximum clock frequency are not reported.

Index terms: RISC-V, RV32I, soft-core processor, datapath, register file, ALU, VHDL, FPGA.

1. Objective

The objective was to implement an instruction-set architecture at register-transfer level and confirm that the hardware honours its architectural rules: immediate encoding, the x0-is-zero rule and control-flow redirection. RISC-V was chosen for its open specification and regular 32-bit encoding, which simplifies decoding [2].

Table 1. Architectural specification.

Property	Value
ISA	RV32I subset (Table 2)
Organisation	Single-cycle
Data width	32 bits
Register file	32 x 32 bits; 2 asynchronous read ports, 1 synchronous write port; x0 = 0
Control	Combinational decode of opcode, funct3, funct7
Target	FPGA development board

2. Instruction Subset

All instructions share fixed field positions: opcode [6:0], rd [11:7], funct3 [14:12], rs1 [19:15], rs2 [24:20], funct7 [31:25]. The six formats (R, I, S, B, U, J) differ in immediate construction, which the decoder reconstructs with sign extension.

Table 2. Supported RV32I encodings.

Instruction	opcode	funct3	funct7
ADD, SUB	0110011	000	0000000, 0100000
AND, OR, XOR	0110011	111, 110, 100	0000000
SLL, SRL	0110011	001, 101	0000000
SLT	0110011	010	0000000
ADDI	0010011	000	N/A
LW, SW	0000011, 0100011	010	N/A
BEQ, BNE	1100011	000, 001	N/A
JAL, LUI	1101111, 0110111	N/A	N/A

3. Datapath and Control

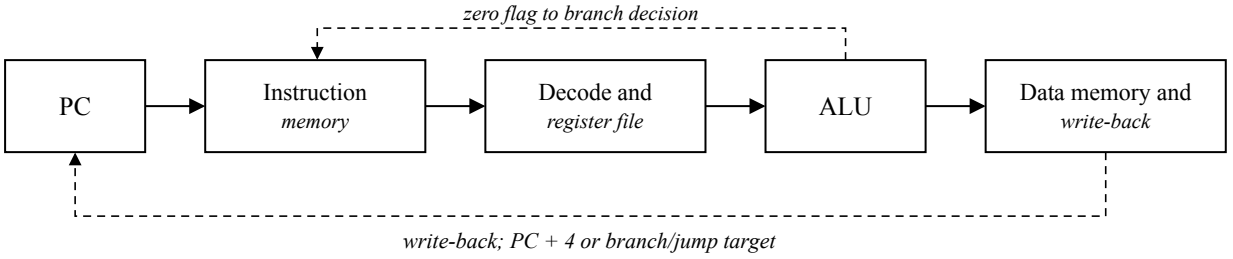


Figure 1. Single-cycle datapath. In one clock period the PC addresses instruction memory, the decoder drives the register file and control signals, the ALU computes, and the result is written back. The PC advances by 4 or is redirected to a target formed from the sign-extended immediate.

Register file. Reads are asynchronous and writes occur on the rising clock edge. Writes with rd = 0 are discarded and x0 reads as zero, so the rule is enforced structurally.

ALU. Addition and subtraction share one adder, with subtraction by two's-complement negation. The ALU also implements AND, OR, XOR, signed set-less-than and logical shifts. A zero flag allows BEQ and BNE to be resolved in the same cycle.

Control. The control unit maps opcode, funct3 and funct7 to register-write enable, ALU operation, operand source (register or immediate), memory read and write, and write-back source. JAL writes PC + 4 to rd through the common write-back path.

4. Verification

Stage 1: RTL simulation. Self-checking testbenches in EDA Playground exercised register-file reads and writes, ALU results and instruction sequencing; waveforms were inspected cycle by cycle against hand-computed values. This localises faults to a specific signal and cycle.

Stage 2: architectural tests. Hand-assembled programs were loaded into instruction memory and the final register state compared with expected values (Table 3). The terminating self-branch holds the machine in a defined state for inspection.

Table 3. Architectural test program and expected end state.

Instruction	Operation	Expected	Simulation	FPGA
addi x1, x0, 5	Immediate add	x1 = 5	Pass	Pass
addi x2, x0, 7	Immediate add	x2 = 7	Pass	Pass
add x3, x1, x2	Register add	x3 = 12	Pass	Pass
sub x4, x2, x1	Subtract	x4 = 2	Pass	Pass
and x5, x2, x1	Bitwise AND	x5 = 5	Pass	Pass
slt x6, x1, x2	Set less than	x6 = 1	Pass	Pass
beq x0, x0, loop	Branch (taken)	PC held	Pass	Pass

The documented program exercises ADDI, ADD, SUB, AND, SLT and BEQ. The remaining instructions in Table 2 are implemented in the decoder and ALU; their individual test results are not documented in this report.

Stage 3: implementation. The verified RTL was synthesised to LUTs and flip-flops, placed and routed, and checked by static timing analysis against the clock constraint. The board then executed the test program.

5. Design Trade-off

A single-cycle organisation keeps the mapping from architecture to logic direct, at the cost of a clock period set by the slowest instruction's full datapath traversal. Table 4 compares it with a five-stage pipeline [1].

Table 4. Single-cycle and pipelined organisations.

Property	Single-cycle (implemented)	Five-stage pipeline
Ideal CPI	1	1 plus hazard stalls
Clock period set by	Full datapath	Slowest stage
Hazards	None	Data, control, structural
Verification	Directed tests	Directed tests plus forwarding and stall coverage

$$CPI = 1 + f_{branch} \cdot p_{mispredict} \cdot penalty + f_{load\text{-}use} \tag{1}$$

6. Limitations

- FPGA device, resource utilisation (LUTs, flip-flops, block RAM), maximum frequency and timing slack are not reported.
- Instruction coverage is documented only for the six instructions in Table 3.
- The core has not been run against the RISC-V architectural test suite or formally verified.
- Unsigned set-less-than (SLTU) and arithmetic shift (SRA) are not implemented.

7. Conclusion and Future Work

The core executes an RV32I subset correctly in simulation and on hardware for the documented tests. Future work: report utilisation and F_{max}; extend directed tests to every implemented instruction; run the riscv-arch-test compliance suite and riscv-formal bounded model checking; and implement a five-stage pipeline with forwarding, load-use stalls and branch prediction.

Appendix A. RTL Excerpt

```
architecture rtl of alu is
  signal r : std_logic_vector(31 downto 0);
begin
  process(a, b, alu_op)
    variable sa : integer range 0 to 31;
  begin
    sa := to_integer(unsigned(b(4 downto 0)));
    case alu_op is
      when "0000" => r <= std_logic_vector(unsigned(a) + unsigned(b));      -- ADD
      when "0001" => r <= std_logic_vector(unsigned(a) - unsigned(b));      -- SUB
      when "0010" => r <= a and b;                                          -- AND
      when "0011" => r <= a or b;                                           -- OR
      when "0100" => r <= a xor b;                                          -- XOR
      when "0101" => r <= std_logic_vector(shift_left(unsigned(a), sa));    -- SLL
      when "0110" => r <= std_logic_vector(shift_right(unsigned(a), sa));   -- SRL
      when "0111" => r <= (0 => '1', others => '0') when signed(a) < signed(b)
                          else (others => '0');                             -- SLT
      when others => r <= (others => '0');
    end case;
  end process;
  result <= r;
  zero   <= '1' when r = x"00000000" else '0';
end architecture;

-- register file write port: writes to x0 discarded
process(clk) begin
  if rising_edge(clk) then
    if we = '1' and rd /= "00000" then x(to_integer(unsigned(rd))) <= wd; end if;
  end if;
end process;
rd1 <= x(to_integer(unsigned(rs1)));
rd2 <= x(to_integer(unsigned(rs2)));
```

Listing A1. ALU and register-file write logic (entity declarations omitted).

References

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: RISC-V Edition*, 2nd ed. Morgan Kaufmann, 2020.

[2] A. Waterman and K. Asanović, Eds., *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*. RISC-V International, 2019.

[3] S. L. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. Morgan Kaufmann, 2021.

[4] P. J. Ashenden, *The Designer's Guide to VHDL*, 3rd ed. Morgan Kaufmann, 2008.

[5] *IEEE Standard VHDL Language Reference Manual*, IEEE Std 1076-2008, 2009.